Robotics and Autonomous Systems 74 (2015) 51–65

Contents lists available at ScienceDirect

Robotics and Autonomous Systems

journal homepage: www.elsevier.com/locate/robot

A novel method for computation of importance weights in Monte Carlo localization on line segment-based maps

Biswajit Sarkar [a,*], Surojit Saha [a], Prabir K. Pal [b]

[a] Computer & Informatics Group, Variable Energy Cyclotron Centre, 1/AF, Bidhan Nagar, Kolkata 700 064, India
[b] Division of Remote Handling & Robotics, Bhabha Atomic Research Centre, Trombay, Mumbai 400 085, India

HIGHLIGHTS

- Monte Carlo localization has rarely been studied on line segment-based maps.
- Importance weights play a key role in the performance of Monte Carlo localization.
- A heuristic-driven method for weight computation on segment-based maps is proposed.
- The proposed method is compared with three other weight computation methods.
- Results corroborate that the proposed method is more accurate, robust and efficient.

ARTICLE INFO

Article history:
Received 22 August 2014
Received in revised form
22 June 2015
Accepted 3 July 2015
Available online 10 July 2015

Keywords:
Importance weight
Line-segment map
Localization
Monte Carlo
Particle filter
Pose tracking

ABSTRACT

Monte Carlo localization is a powerful and popular approach in mobile robot localization. Line segment-based maps provide a compact and scalable representation of indoor environments for mobile robot navigation. But Monte Carlo localization has seldom been studied in the context of line segment-based maps. A key step of the approach – and one that can endow it with or rob it of the attributes of accuracy, robustness and efficiency – is the computation of the so called importance weight associated with each particle. In this paper, we propose a new method for the computation of importance weights on maps represented with line segments, and extensively study its performance in pose tracking. We also compare our method with three other methods reported in the literature and present the results and insights thus gathered. The comparative study, conducted using both simulated and real data, on maps built from real data available in the public domain clearly establish that the proposed method is more accurate, robust and efficient than the other methods.

© 2015 Elsevier B.V. All rights reserved.

1. Introduction

The capability of localization, which enables a mobile robot to continually estimate its position and heading direction (collectively referred to as *pose*), is a key requirement for autonomous navigation. While odometers do provide incremental estimates of pose, such estimates cannot be trusted upon for long periods of time because of integration of measurement errors over the length of travel. Modern mobile robots are equipped with Laser Range Finders (LRFs), which provide the robot with an accurate range estimate of the nearby objects. Thus, if a spatial model of the environment is available in the form of a map, it is possible to estimate the pose of the robot.

Maps are commonly represented as occupancy grids [1]. A major drawback of the occupancy grid representation is that a large amount of memory is necessary to store the map. The requirement of memory increases as the size of the environment to be mapped increases. The recent trend of deploying mobile robots in larger and larger working environments has made the issue of scalable map representations a significant one. Maps based on line segments [2–4] are compact, provide floating-point resolution, consume significantly less memory and scale well with the environment size. Moreover, line segments are an obvious choice in the representation of indoor environments that predominantly comprise objects like walls, corridors and cupboards. These objects have planar external surfaces and thus naturally lend themselves to representation by line segments in the 2D-plane.

Monte Carlo Localization (MCL) [5] is a popular approach for mobile robot localization. It employs a Bayes' filter to recursively estimate the posterior probability density function of the robot

* Corresponding author. Tel.: +91 33 2318 2224; fax: +91 33 2334 6871.
E-mail address: bsarkar@vecc.gov.in (B. Sarkar).

http://dx.doi.org/10.1016/j.robot.2015.07.001
0921-8890/© 2015 Elsevier B.V. All rights reserved.

pose, over the state space, given (noisy) measurements that arrive sequentially in time and probabilistic models of robot motion and perception. It uses a finite set of random samples (a.k.a *particles*) with associated *importance weights* (or simply, *weights*) to represent, in approximation, the posterior density. Each particle, representing a robot pose, is a hypothesis as to the true, but unknown pose, while its associated weight provides a relative measure of the "importance" of the sample towards approximation of the target density function. Hereinafter, we shall use the terms sample, particle and hypothesis synonymously.

MCL makes no restrictive assumption about the state space dynamics or the shape of the density function. It is able to track multiple hypotheses about the robot pose and can also trade-off the computational resources available with the accuracy of localization. It has a modular structure and is simple to implement. The benefits associated with the use of line segment-based maps (as detailed above) and the effectiveness of MCL as evident, for instance, from [6], are sufficient motivations for implementing MCL on such maps. Unfortunately, studies on the performance of MCL on line segment-based maps have not been extensively reported in the literature. To the best of our knowledge, only [7] has reported it in some detail. A few other authors [8,9] do talk about MCL on such maps, but they are mentioned in different contexts.

The particle filtering framework on which MCL is based only lays down the broad sequence of steps to be followed. The choice of the motion model and the measurement model is left to the programmer. The importance weights, derived from the measurement model, play a key role in the pose estimation process and thus it is imperative to compute the weights in a judicious and efficient manner.

In this paper, we present the results and insights gathered from our investigation into different methods of weight computation in the context of line segment-based maps. The main theme of this paper, however, relates to a proposal on a new, heuristic-driven approach for weight computation on such maps. Apart from being simple to implement and efficient to compute, the novelty in our approach lies in the fact that it just takes into account how closely the actual perception of the robot fits in or is consistent with the environment map when referred to a hypothesized pose. We extensively study the performance of a Monte Carlo-based method for pose tracking that incorporates the proposed approach of weight computation, both using simulated data and real data. Detailed comparative study reveals that our method is more efficient, robust and accurate than three other methods [7,10,11]. The studies were conducted in three different environments whose maps were built from real data using the method described in [2].

The remainder of the paper is organized as follows: In Section 2, we make a brief review of related work. In Section 3, we describe the proposed method, while in Section 4, we overview three competing methods for weight computation. In Section 5, results obtained from simulated and from real data are presented while in Section 6, we discuss on issues germane to the results obtained in Section 5. We summarize the concluding remarks in Section 7.

2. Related work

Mobile robot localization has been an area of active research for around three decades. A myriad of localization methods, differing in approach as well as in the intended domain of application, has been proposed during these years. When formulated as a probability density estimation problem, localization is commonly solved using the Bayes' filter [12,13]. The Kalman filter-based methods postulate the robot pose as a unimodal Gaussian distribution and assume the state-transition model and the measurement model to be linear in its arguments with added Gaussian noise. However, in real-life applications, the Extended Kalman filter (EKF) is often used to address deviation from linearity [14,15].

Methods based on Markov Localization (ML) maintain a discrete approximation of the probability density function of the robot's pose. If the state space is represented as a grid array, as in [16–18], each cell of the grid holds the probability that the robot is currently located in it. Experimental evidence suggests that while grid-based ML is more robust, Kalman filter-based methods are more efficient and accurate [19]. To inherit the virtues of both, a hybrid method named Markov–Kalman localization (ML-EKF) was proposed in [20].

MCL utilizes a particle filter-based framework [21] to overcome the computational overhead and memory demands inherent in grid-based ML methods by using a sample-based representation of the probability density [5,22]. By using two different ways of generating samples, a more robust method that goes by the name of Mixture-MCL was proposed in [23]. A method that improves upon the efficiency of MCL by constraining the robot's path such that it suffices to sample the pose space only in the vicinity of the path was proposed in [24]. A method for Monte Carlo-based pose tracking was proposed in [25]. To make the method robust against unmodelled objects in the map, an approach called *sampling from a noncorrupted window* was proposed. A hybridization of ML with MCL, wherein the former is used to zero in on a probable region of the state space before the latter is used to precisely estimate the robot pose in that region, was reported in [26]. In [27], the accuracy of MCL-estimated pose at pre-decided locations was improved upon by matching scans with previously stored reference scans taken at those locations.

The efficacies of the Bayesian methods have often been augmented by combining them with evolution-based methods [8,28–30]. The evolutionary methods are formulated as an optimization problem that progressively reduces the initial uncertainty of the robot pose by generating new solutions in the promising regions of the state space by using feedback information available during the evaluation of the solutions. Localization methods based solely on evolution-based metaheuristics have also been proposed. For instance, Differential Evolution-based algorithms for localization are reported in [31–34].

The method due to Espinace et al. [7] deals with MCL on line segment-based maps. It extracts line segments from the current range scan of the robot. This set of segments is called *Observed Segments Group* or OSG. Next, the portions of the line segments of the map, as visible from a given sample pose (visible in full or in part) are brought together to form a set of line segments, called the *Map Segments Group* or MSG. The method then computes a modified form of the Hausdorff distance [35] between OSG and MSG, followed by a nonlinear transformation of the distance to determine the importance weight associated with the sample pose. In [10] a method is presented for assigning weight to a particle by assessing the Euclidean distance between the laser range scan actually acquired by the robot and the predicted scan obtained from the particle. In the context of presenting an improved resampling technique of a particle filter for tackling the global localization problem, Gasparri et al. [11] proposed a method of weight computation that is similar in concept to the method of [10] but it uses an additional nonlinear transformation to map the computed Euclidean distance values to the corresponding weight values. The method in [7] was improved upon in a recent work [36] by using a different distance measure [37] and using the same nonlinear transformation as used in [11].

Ref. [38] presents a Monte Carlo localization algorithm that works with 3D laser range scans and a line segment-based map. The 3D point cloud is first reduced to a virtual 2D scan, which is then used to compute the weights of the particles by comparing the predicted scans with the virtual scan. Reference to MCL on line

1. Generate particles, $\mathbf{x}_0^{(i)},\ i = 1, \ldots, N$, on or around the known initial pose and set $t \leftarrow 1$
2. For $i = 1, \ldots, N$, propagate the particles by sampling from the motion model:
$$\tilde{\mathbf{x}}_t^{(i)} = \text{sample_motion_model}\left(\mathbf{x}_{t-1}^{(i)}, \mathbf{u}_t\right)$$
given the control action $\mathbf{u}_t$ with added Gaussian noise.
3. For $i = 1, \ldots, N$, compute the importance weight $\tilde{w}_t^{(i)}$ associated with each particle $\tilde{\mathbf{x}}_t^{(i)}$
4. Normalize the importance weights as: $w_t^{(i)} = \frac{\tilde{w}_t^{(i)}}{\sum_{k=1}^{N} \tilde{w}_t^{(k)}}$, for $i = 1, \ldots, N$
5. Compute the pose estimate μ_t as: $\mu_t = \sum_{i=1}^{N} w_t^{(i)} \tilde{\mathbf{x}}_t^{(i)}$
6. Resample with replacement N particles $\left\{\mathbf{x}_t^{(i)}\right\}_{i=1}^{N}$, from the set $\left\{\tilde{\mathbf{x}}_t^{(i)}\right\}_{i=1}^{N}$ based on their importance weights.
7. Set $t \leftarrow t + 1$ and go to step 2.

Fig. 1. Monte Carlo localization algorithm for pose tracking.

segment-based maps without adequate details may also be found in [8,9].

3. The proposed method

The MCL method incorporating the proposed approach for importance weight computation assumes that a line segment-based 2D map, M, of the environment is available *a priori*. The map is built in an initial *offline* phase by merging line segments extracted from registered laser range scans collected from many different scan poses (please see [2] for details). The map M is a set of $|M|$ line segments, $M = \{L_1, L_2, \ldots, L_{|M|}\}$, where each segment L_i $(1 \leq i \leq |M|)$ has a specific orientation, defined by its start point and end point. We further assume that the robot is equipped with a LRF and keeps collecting laser range data while it moves around in its environment.

In the remainder of this section we first describe the general flow of an MCL method, and then describe the proposed method of weight computation.

An MCL method (Fig. 1) starts off with a set of N particles $\left\{\mathbf{x}_0^{(i)}\right\}_{i=1}^{N}$ placed on or around the initial pose, assuming the pose to be known (Step 1). This set of particles is then propagated on the basis of the *control action* $\mathbf{u}_t$. The control action is estimated using odometry and inflated with artificially added Gaussian noise. In practice, the propagated samples at a given time index are obtained by sampling from an appropriate motion model using the particles in the previous time index and the control action (Step 2). The motion model for sampling that we adopt in our work has been borrowed from Section 5.4 of [13]. In Step 3, the importance weight associated with each propagated particle is computed using the proposed weight computation method. Each of the computed weights is then normalized so that they sum up to unity (Step 4). The pose of the robot is estimated in Step 5 by computing the sum of the particles, where each particle is weighted by its associated (normalized) importance weight. The particles are resampled in Step 6 – where the probability of selection of a particle is equal to its importance weight – using the method of *Stochastic Universal Sampling* [39] so that the resulting distribution of particles represents, in approximation, the posterior probability density of the robot pose, given all observations till the current time index [40–42]. This marks the end of one pass in the pose tracking cycle. The time index is subsequently incremented and the cycle is repeated from Step 2 onwards (Step 7).

The weight computation procedure, corresponding to Step 3 of Fig. 1, commences with the extraction of line segments from the laser range scan acquired by the robot from a given scan pose. We use the same procedure for line segment extraction as in our work [2]. We refer to each line segment extracted from a single range scan as a *scan line segment*, or a *scan segment* for short. It is important to note that these segments are all directed, their sense being identical to that in which the LRF obtained the range data from which these segments have been extracted. A comparative study on different methods for line extraction from 2D range data is available in [43].

In every cycle of pose tracking, the scan line segments are bracketed together first with one particle, then with a second and so on, for all particles one after another. The scan line segments are then transformed to the world coordinate system – in which the particles and the environment map are defined – using the pose of the associated particle with which it has been bracketed. Hereinafter, all references to scan segments shall imply scan segments in the world coordinate system defined with respect to the associated particle. If the particle coincides with the true (but unknown) pose of the robot, all the scan segments broadly match with and overlap on the map segments. Otherwise, the degree of mismatch between the scan segments and the corresponding map segments by and large depends on the extent to which the pose of the particle differs from the true pose. The weight computing method that we propose here is based on the above heuristic; by assessing the extent of mismatch, we estimate the discrepancy between the corresponding particle and the true pose.

The weight computation method consists of three steps. In the first step, we establish the correspondence between a scan line segment and a map segment in the sense that both refer to the same physical object in the environment [44,45]. In the second step, we assess the mismatch between a set of scan segments and the set of corresponding map segments. In the third step, we subject the estimate of mismatch to a nonlinear transformation to arrive at the corresponding value of the importance weight. These weights are then normalized in a subsequent step (Step 4 of Fig. 1).

In the proposed method, the measure of mismatch, as well as the way to establish the correspondence, rests on the notions of *lateral offset* and of *longitudinal offset* between two directed line segments. The lateral offset, d_{lat}, of segment $\vec{AB}$ from $\vec{CD}$ is defined as the arithmetic mean of the (absolute) lengths of the perpendiculars from A and B on the line supporting $\vec{CD}$. The overhead arrows are used to emphasize that the line segments are directed and have a specific sense of orientation. Referring to Fig. 2,

$$d_{\text{lat}}\left(\vec{AB}, \vec{CD}\right) = \frac{AX + BY}{2}$$

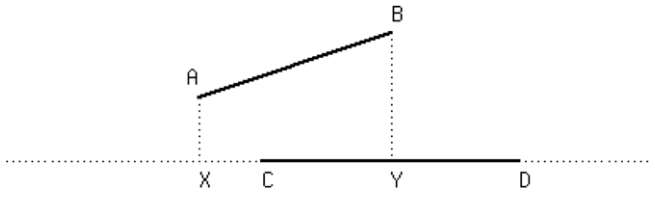


Fig. 2. The lateral offset and the longitudinal offset of $\vec{AB}$ from $\vec{CD}$ provide a measure of their mismatch, where $\vec{AB}$ represents a scan segment and $\vec{CD}$ a map segment.

where X and Y are the feet of the perpendiculars from A and B respectively on the line supporting $\vec{CD}$. It is important to note that the lateral offset implicitly measures the angular deviation between the segments $\vec{AB}$ and $\vec{CD}$.

The longitudinal offset, d_{long}, of segment $\vec{AB}$ from $\vec{CD}$ is defined as the amount of translation required on the projection $\vec{XY}$ of $\vec{AB}$ along the line supporting $\vec{CD}$ such that $\vec{XY}$ is completely subsumed in $\vec{CD}$. Referring to Fig. 2, $d_{\text{long}}\left(\vec{AB}, \vec{CD}\right) = XC$. If the position and orientation of $\vec{AB}$ were such that its projection totally overlapped with $\vec{CD}$, the longitudinal offset would have been zero. Here, we tacitly assume that the lengths and orientations of $\vec{AB}$ and $\vec{CD}$ are such that it is possible to totally subsume $\vec{XY}$ in $\vec{CD}$. We also assume that the points A and C are identical in the sense that both of them are either the start points or the end points of their corresponding segments. Here, it is important to note that these offset measures are not symmetric with respect to their arguments. Even though both are line segments, a map segment (a representative of ground truth) is intrinsically different from a scan segment (a noisy and partial view of the ground truth). Hence, the arguments to the above measures cannot be interchanged.

Let us denote the set of scan line segments associated with a given particle by S and let s_j $(1 \leq j \leq |S|)$ be an individual scan segment such that $s_j \in S$. Similarly, let m_i $(1 \leq i \leq |M|)$ denote an individual map line segment and M the set of all map segments, such that $m_i \in M$. It is to be kept in mind that all s_j's and m_i's above are directed and have a specific sense of orientation. If $s_p \in S$ and $m_q \in M$ refer to the same physical planar surface of the environment, then s_p will, ideally, not be greater than m_q in length. This is so because in a specific scan only a part of, or at best the entire, planar surface modelled by a map segment will be visible. To identify which scan segment corresponds to which map segment, we first compute the lateral and the longitudinal offsets between a given scan segment and each of the map segments. The pseudo codes for computing these offsets are given in Fig. 3. We assume that the function average(.) returns the arithmetic mean of the two arguments passed to it, while start(.) and end(.) return the start point and the end point respectively of the directed line segment passed on to them as argument.

At the heart of the weight computation method lies the function named AssessMismatch(.) (Fig. 4). It is invoked separately for each particle and takes in S and M as its arguments and returns an estimate of mismatch between S and the corresponding elements of M. Thus, the return value of AssessMismatch(.) gives us an idea of the extent to which the particle is "away" from the true, but unknown, pose.

For each scan line segment $s_j \in S, j = 1, \ldots, |S|$ we determine (i) whether a given map segment $m_i \in M$, $i = 1, \ldots, |M|$ lies within an angle of $\frac{\pi}{6}$ to s_j; and (ii) whether s_j is less than or equal to m_i in length (lines 8 – 9). If these two conditions are satisfied, m_i is a possible candidate that corresponds to s_j. The specific value of $\frac{\pi}{6}$ in (i) above is empirically chosen and it ensures that if a given m_i differs widely ($\frac{\pi}{6}$ or beyond) from s_j in orientation, it is not to be considered for possible correspondence with that s_j. To determine the specific m_i (let us call it m_i^*) from among the candidate segments that we believe actually corresponds to s_j, we compute the sum of the lateral offset and the longitudinal offset between s_j and all candidate m_is one after another, and the m_i for which the sum is least is reckoned as the map segment, m_i^* corresponding to s_j. In case the two conditions (in lines 8–9) are not satisfied for a given scan segment, we reckon that no corresponding map segment for it exists and proceed with the next scan segment. If no scan segment from the set S corresponds to any map segment in M, the Boolean variable flag retains its initial value of *false*. This implies that if the current pose of the robot is supposed identical to that of the particle under consideration, the present perception of the robot is not consistent with the environment. This scenario is signalled by returning a value of -1 (line 24). If

```
1 //Computes the distance of a given point from the line supporting a given line segment
2 Function d(point: p, line segment: l)
3     return(perpendicular distance of p from the line supporting l)
4
5
6 //Computes the lateral offset of a scan line segment from a map segment
7 Function d_lat(scan segment: s, map segment: m)
8     return ( average( d(start(s),m), d(end(s),m) ) )
9
10
11 //Computes the longitudinal offset of a scan line segment from a map segment
12 Function d_long(scan segment: s, map segment: m)
13    s'=projection of s on the line supporting m
14    if s' is totally contained in m
15         return (0)
16    else
17         return (amount of translation of s' required along the
18         line supporting m so that s' is totally contained in m)
```

Fig. 3. Pseudo code for computing the lateral and the longitudinal offsets of a scan line segment from a map segment.

```
1  Function AssessMismatch(scan segment set: S, map segment set: M)
2  Δ = 0
3  Len = 0
4  flag = FALSE
5  for j = 1,2,....., |S| do
6     D = δ = infinity
7     for i=1,2,....., |M|
8        if ((angle between s_j∈S and m_i∈M is less than π/6)
9           AND (s_j is not greater than m_i in length))
10             flag = TRUE
11             lateral = d_lat(s_j,m_i)
12             longitudinal = d_long (s_j,m_i)
13             D = lateral + longitudnal
14             if (D < δ )
15                δ = D
16             endif
17       endif
18    endfor
19    if (δ < infinity)
20       Δ = Δ + δ*length(s_j)
21       Len=Len + length(s_j)
22    endif
23 endfor
24 if (flag = FALSE) return (-1)
25 Δ = Δ/Len
26 return (Δ)
```

Fig. 4. Pseudo code for computing the mismatch between a set of scan line segments and the map line segments.

the corresponding map segment could be found for at least one scan segment – indicated by the value *true* of flag (set in line 10) – the total mismatch between the sets S and M is defined as the weighted average of the offsets of each scan line segment from its corresponding map line segment weighted by the length of the scan line segment (lines 20–21 and 25) and it is this mismatch that is returned in line 26. We assume that length(.) returns the length of the segment passed on to it as argument.

After the mismatch between a set S and M has been estimated, the particle associated with S is assigned weight using the function ImportanceWeight(.) that takes in the estimate as its argument (Fig. 5). The weight corresponding to a mismatch of 'Δ' is given by Δ^{-2} (line 7), provided of course that the mismatch is not -1 or is not less than 1. In the former case, the weight is assigned a value of 0 (line 3) while in the latter case, it is assigned a fixed value of 1 (line 5).

4. Competing methods for weight computation

In this section, we briefly describe three other methods for weight computation that compete with the proposed method. These methods are separately used to realize Step 3 of the MCL method given in Fig. 1 and the results of pose estimate compared with that of our method.

The method of Yaqub and Katupitiya [10] computes the normalized weight of a particle 'i' $\{i = 1, 2, \ldots, N\}$ as follows:

$$w_i = \frac{1}{N-1}\left[1 - \frac{\sum_{j=1}^{n}\left(\rho_j - R_j^{(i)}\right)^2}{\sum_{k=1}^{N}\left\{\sum_{j=1}^{n}\left(\rho_j - R_j^{(k)}\right)^2\right\}}\right] \tag{1}$$

where ρ_j is the actual range sensed by the jth beam of the LRF of the robot during acquisition of the scan, while $R_j^{(k)}$ is the (predicted) range obtained by the kth particle in the corresponding direction. The predicted range value in a given direction from a particle is computed by ray tracing. We assume that there are n range points in a scan and a total of N particles.

In the method of Gasparri et al. [11] the weights are computed as follows:

$$w_i = \frac{\beta}{\sum_{j=1}^{n}\left(\rho_j - R_j^{(i)}\right)^2} \tag{2}$$

where β is a normalization constant, whose specific value is unimportant because of subsequent normalization (Step 4 of Fig. 1).

```
1  Function ImportanceWeight(mismatch: Δ)
2  if (Δ = -1)
3    weight = 0
4  else if (Δ < 1)
5    weight = 1
6  else
7    weight = 1/(Δ)^2
8  endif
9  return (weight)
```

Fig. 5. Pseudo code for transforming an estimate of mismatch to importance weight.

The method of Espinace et al. [7] first rotates a particle and its associated MSG so that the MSG roughly aligns with OSG (please see Section 2) and then computes the Modified Hausdorff Distance (MHD) [35] between OSG and the rotated MSG for each of the rotated N particles. The weight associated with the ith particle is then computed as

$$w_i = \frac{1 - \tanh\left(\frac{2*\left((\mathrm{MHD}(\mathrm{MSG}_i, \mathrm{OSG})) - l_p\right)}{l_p}\right)}{2} \tag{3}$$

where MSG_i is the set of map segments likely to be perceived by the ith particle (in entirety or in part), the function MHD(.) computes the MHD between the two arguments passed on to it and l_p is a parameter that regulates how fast the values of the weights decay with MHD. The MSG is computed by ray tracing. In our implementation, we chose the value of l_p in such a way that a particle, which is away from the true pose by half the distance of the particle farthest from the true pose, is assigned a weight of 0.5. With other values e.g., $l_p = 0.2$, the performance of the method was found to deteriorate as we shall see later.

5. Simulation and experimental results

In this section we report on the results obtained from simulation [46] and experimental studies conducted using four different MCL methods. For ease of reference, henceforth, we shall refer to MCL incorporating a specific weight computation method by the first letter of the surname of the first author proposing the method. Thus, MCL-E refers to MCL incorporating the weight computation method proposed in [7]. Similarly, MCL-Y and MCL-G refers to MCL incorporating the methods [10,11] respectively. MCL-S refers to MCL incorporating the proposed weight computation method.

The studies were carried out to evaluate and compare different aspects of the effectiveness and efficiency of the proposed MCL-S method with the three competing methods (MCL-E, MCL-Y and MCL-G) in pose tracking. The source codes that we developed for the four methods were identical, except for the step of weight computation (step 3 of Fig. 1). The difference in only this step ensured that the variation in average performance among the four methods arose essentially due to the difference in that step. The studies were conducted in the three different indoor environments, hereinafter referred to as Env#1, Env#2 and Env#3. The maps of these environments were built using the method reported in [2] based on the real-life datasets available in the public domain at Radish [47] and mentioned against each environment given below.

- Env#1: SRI AIC K wing ($\sim$ size: 83 m $\times$ 21 m);
- Env#2: Department of DIIGA, Engineering University, Ancona ($\sim$ size: 47 m $\times$ 47 m); and
- Env#3: A 375 m long corridor of the main building of Chosun University.

(i) *Pose estimation using simulated data*

Each map of the environment was loaded in a simulator, and a simulated robot driven along a pre-specified path on the map. While the robot moved, laser range data and the corresponding true scan poses (obtained from the simulator) were recorded. The true scan poses were then perturbed by injecting Gaussian noise and the resulting values reckoned as the corresponding odometry estimates of the poses. (Hereinafter, all references to odometry-estimated poses in the simulated environment shall imply estimates of the poses obtained in the above way.) Thereafter, only the odometry estimates of the scan poses and the range data acquired from those poses were made available to the robot for localization. The poses, as estimated by the localization methods, were then compared with the corresponding true poses for determination of the errors in the pose estimates.

For injecting noise, the odometry motion model given in Section 5.4 of [13] was used. The noise parameters (standard deviation of Gaussian distribution) are represented as $\sigma_{\Delta d,d}$, $\sigma_{\Delta\theta,\theta}$, $\sigma_{\Delta\theta,d}$, and $\sigma_{\Delta d,\theta}$ with units mm/m, deg/360°, deg/m and mm/degree respectively. Using a different seed every time, 30 different trajectories were generated for each of the three sets of noise parameters (see Table 1 for values), resulting in 90 different trajectories for each environment. The red trajectories in Fig. 6 are instances of odometry-estimated trajectories obtained by injecting noise (with parameters of set Noise#3) to the corresponding true trajectories, indicated in blue, along which the robot was actually driven. The total lengths of paths travelled by the robot were around 500 m, 165 m, and 580 m in Env#1, Env#2 and Env#3 respectively.

For evaluation and comparison of their performances, MCL-E, MCL-Y, MCL-G and MCL-S were separately run with 200 particles on each of the different trajectories. A cycle of MCL (steps 2–7 of Fig. 1) was invoked for pose estimation only when the current pose of the robot differed from the pose on which the previous cycle was invoked by at least 500 mm or 5°. The poses here refer to odometry estimates of scan poses. The Euclidean distance between the true position of the robot and the corresponding position as estimated by the localization method under evaluation was computed for each scan pose. In a similar fashion, the angular distance between the true orientation and the estimated orientation was also computed for each scan pose. We refer to these distances as the *position error* and the *orientation error* incurred in the estimation

Table 1
Three different settings of the noise parameters used in our study.

	$\sigma_{\Delta d,d}$ (mm/m)	$\sigma_{\Delta\theta,d}$ (degree/m)	$\sigma_{\Delta\theta,\theta}$ (degree/360°)	$\sigma_{\Delta d,\theta}$ (mm/360°)
Noise #1	10	5	5	2
Noise #2	100	10	10	4
Noise #3	200	20	20	10

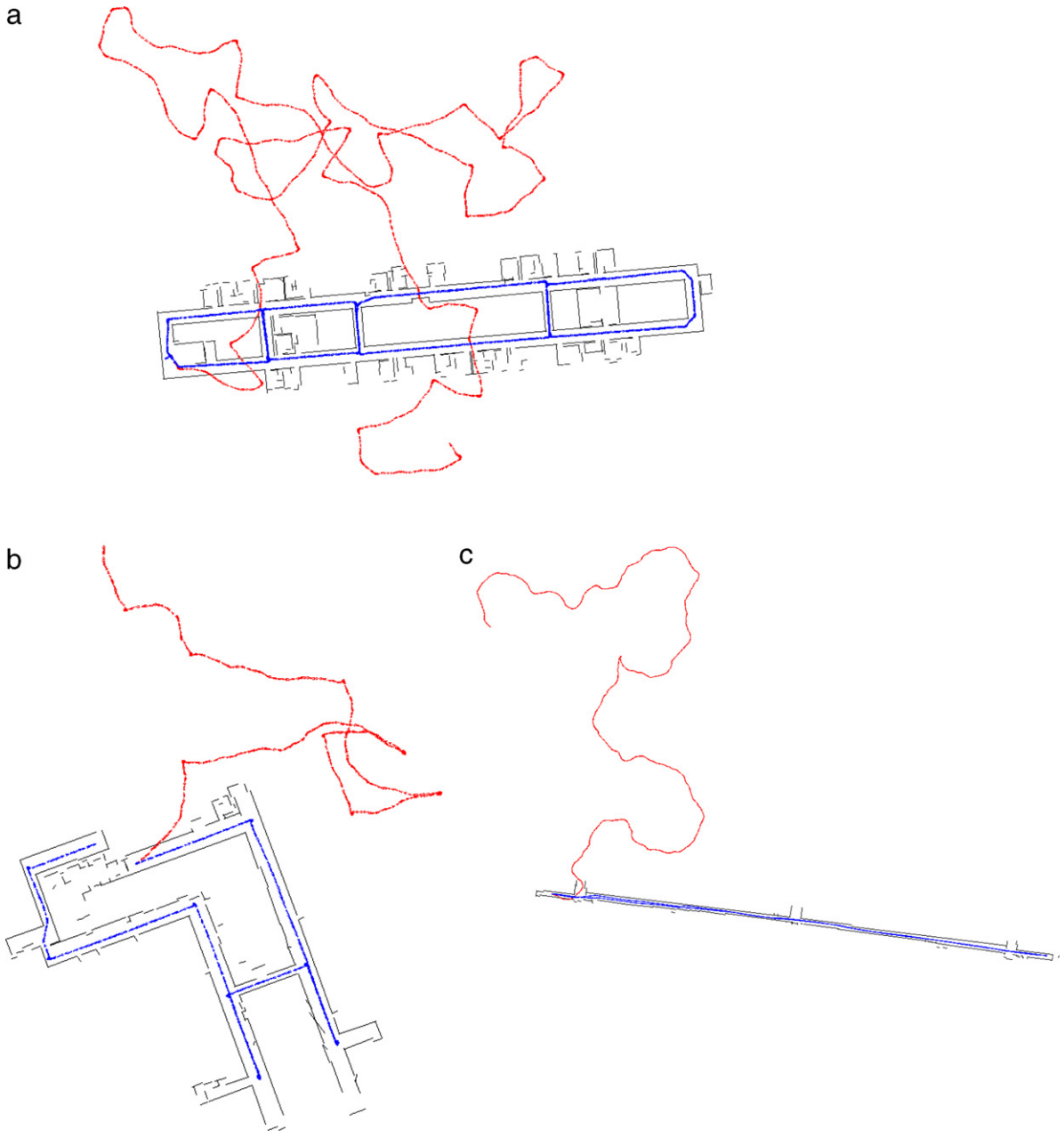


Fig. 6. The blue trail shows the path along which the robot was actually driven, while the trajectory reconstructed by adding Gaussian noise with parameters corresponding to Noise#3 is shown in red in (a) Env#1; (b) Env#2; and (c) Env#3. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

of each scan pose respectively by the localization method under evaluation. When these errors are averaged over all scan poses in a given run along the entire trajectory, we refer to the resulting quantities as the *average position error* and the *average orientation error* respectively for the trajectory incurred by the corresponding localization method. Since these average errors pertain to a single run of a localization method on a particular trajectory, they are not suitable figures-of-merit on which to compare the methods with. So, we consider the mean of the average errors of 30 runs, where each run is taken on a different trajectory that is generated from the same set of noise parameters of Table 1 using a different seed. We also consider the 95% confidence interval of the mean computed using a t-distribution.

In Fig. 7(a) and (b), we show how the mean of the average errors in position and in orientation, as estimated by the four methods, compare against each other for different levels of injected noise in Env#1. It is important to point out that on quite a few occasions MCL-G failed to estimate the poses after a while (we

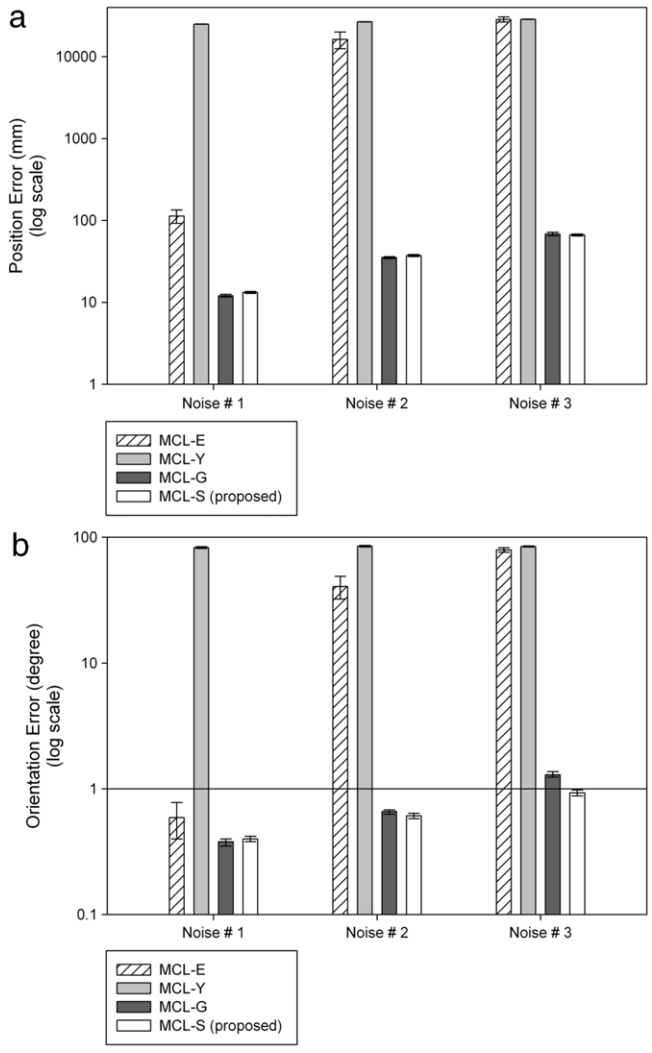


Fig. 7. Comparison of the accuracy of pose estimation for different levels of injected noise in Env#1: (a) the mean of the average position errors; and (b) the mean of the average orientation errors. Both the errors are plotted in log scale. The error bars indicate 95% confidence interval. MCL-G has failed to track poses on 8 occasions out of 30 for Noise#3.

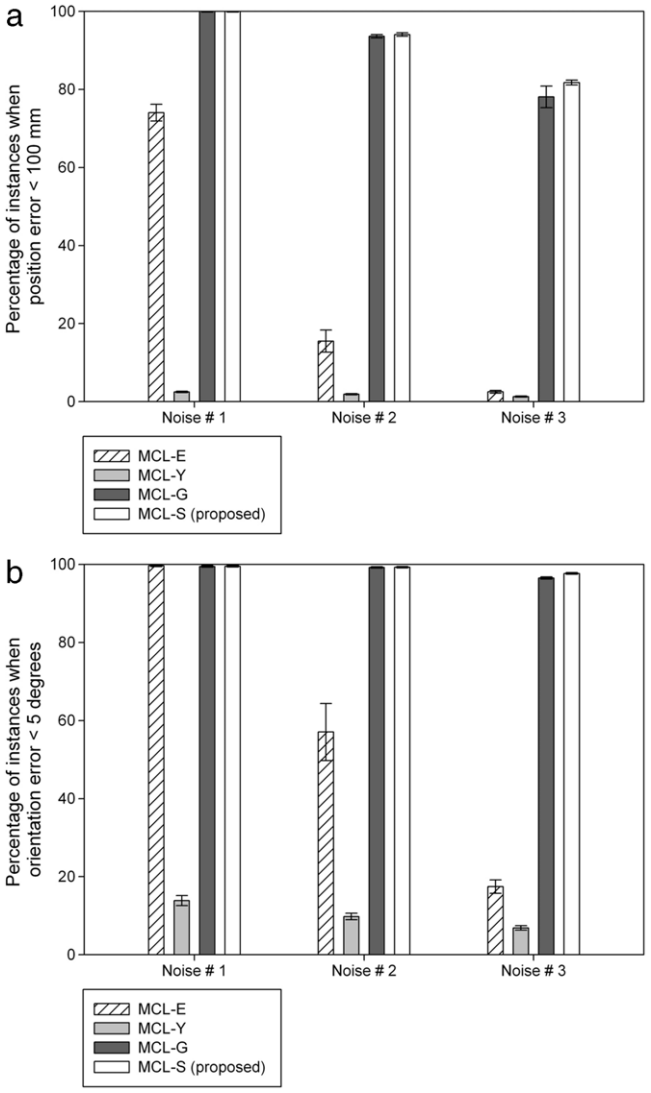


Fig. 8. Comparison of the percentage of occasions when (a) the estimated positions differ from the corresponding true positions by less than 100 mm; (b) the estimated orientations differ from the corresponding true orientations by less than 5°. The error bars represent 95% confidence intervals. The results pertain to Env #1 where MCL-G failed on 8 occasions out of 30 for Noise#3.

discuss the reasons for failure in Section 6). To be specific, MCL-G failed on eight occasions out of 30 attempts for Noise#3. Thus, the mean of the average position and of the orientation errors and the confidence intervals were based on 22(=30 − 8) runs for MCL-G for Noise#3.

From Fig. 7, it is evident that the proposed MCL-S and the MCL-G easily outperform the other two methods insofar as accuracy of pose tracking is concerned. These two methods are roughly at par, even though MCL-S makes a slightly better estimate of the orientation for Noise#3. Though accurate, MCL-G is not adequately robust as it lost track of the robot on eight occasions out of 30 attempts for Noise#3. As stated in Section 4, the above results (as also all other results reported in this paper) for MCL-E were obtained with $I_p = 0.5$. For lower values of I_p, e.g., $I_p = 0.2$, the estimates improved for Noise#1, but deteriorated for Noise#2 and Noise#3 and MCL-E even failed to track poses on a couple of occasions.

In Fig. 8(a) and (b), we show the mean of the number of occasions, expressed as a percentage, when the position errors and the orientation errors in the estimated scan poses were less than 100 mm and less than 5° respectively as the robot traversed its trajectory in Env#1. As in Fig. 7, for each set of noise parameters, every method was run on 30 different trajectories and only those many runs were used in computing the final results as could be successfully completed. The number of scan poses in each trajectory ranged between 1200 and 1550.

From Fig. 8(a) and (b) it is evident that in almost all cases the estimates of the poses differ from the corresponding true poses by less than 100 mm and 5° for MCL-G and the proposed MCL-S for Noise#1. With an increase in the level of injected noise, the number of occasions when the differences are so, drop. But even at Noise#3, the number of such occasions was around 80% (for position estimate) and 96% (for orientation estimate) for MCL-G and MCL-S, though MCL-S was marginally better in both estimates. For the proposed MCL-S, we never encountered a situation when the estimated position differed from the true position by more than 1 m. But, there were a few such occasions for MCL-G with Noise#3. For Noise#1, MCL-E estimated the orientations with less than 5° error in almost all cases. But, it performed poorly for Noise#2 and Noise#3. Irrespective of the levels of injected noise, the performances of MCL-Y were poor. The number of occasions when the position error was more than 1 m was over 90% for MCL-Y for all the three injected noise levels. Thus Fig. 8 serves to attest to the accuracy of both MCL-G and MCL-S from another standpoint.

Table 2
Mean and standard deviation of the average position and the average orientation errors for Env#2 and Env#3. The standard deviation is given in square bracket below the corresponding mean.

Method	Mean [std. dev] of average position errors (mm)			Mean [std. dev] of average orientation errors (deg)			#Failures (in 30 runs)
	Noise#1	Noise#2	Noise#3	Noise#1	Noise#2	Noise#3	
Env#2							
MCL-E	88.07 [22.15]	304.63 [131.90]	18,801.42 [5938.12]	0.32 [0.00]	0.65 [0.55]	72.59 [20.00]	Nil
MCL-Y	15,125.51 [1649.26]	18,481.38 [835.50]	20,430.11 [739.70]	72.19 [11.00]	85.90 [8.99]	93.02 [11.61]	Nil
MCL-G	15.34 [1.89]	46.84 [8.19]	92.81 [15.44]	0.36 [0.01]	0.69 [0.03]	1.45 [0.24]	12 (for Noise#3)
MCL-S	21.17 [3.59]	53.58 [6.24]	91.40 [9.42]	0.57 [0.04]	0.94 [0.07]	1.55 [0.12]	Nil
Env#3							
MCL-E	14,376.82 [29,297.85]	63,268.42 [7827.04]	68,388.76 [10,219.55]	15.35 [31.12]	61.12 [11.39]	62.43 [10.93]	Nil
MCL-Y	81,238.33 [6484.17]	79,533.65 [5159.95]	82,113.65 [5133.06]	80.87 [12.87]	81.26 [10.13]	87.59 [8.47]	Nil
MCL-G	80.64 [52.62]	5407.38 [20,761.18]	–	0.42 [0.04]	7.14 [24.20]	–	4 (for Noise#2) All 30 (for Noise#3)
MCL-S	64.66 [21.47]	237.06 [49.91]	492.98 [316.42]	0.63 [0.04]	1.06 [0.11]	3.68 [0.35]	Nil

Table 3
Percentage of occasions and the standard deviation when (a) the estimated positions differ from the corresponding true positions by less than 100 mm; (b) the estimated orientations differ from the corresponding true orientations by less than 5° for Env#2 and Env#3. The standard deviation is given in square bracket below the corresponding percentage of occasions.

Method	% Instances [std. dev] Position Errors $<$ 100 mm			% Instances [std. dev] Orientation Errors $<$ 5°			#Failures (in 30 runs)
	Noise#1	Noise#2	Noise#3	Noise#1	Noise#2	Noise#3	
Env#2							
MCL-E	64.37 [9.58]	35.20 [5.47]	2.86 [3.71]	100.00 [0.00]	99.31 [0.71]	38.75 [15.41]	Nil
MCL-Y	1.74 [0.58]	1.25 [0.29]	0.77 [0.22]	10.99 [6.32]	6.34 [3.37]	3.08 [2.13]	Nil
MCL-G	99.92 [0.29]	89.65 [3.72]	72.63 [3.31]	99.98 [0.06]	99.55 [0.30]	95.57 [1.06]	12 (for Noise#3)
MCL-S	98.74 [1.65]	85.85 [3.21]	68.43 [3.57]	99.34 [0.46]	97.20 [0.82]	93.01 [1.34]	Nil
Env#3							
MCL-E	20.66 [11.90]	2.05 [1.70]	1.63 [1.16]	91.04 [18.48]	63.13 [6.32]	62.74 [6.30]	Nil
MCL-Y	0.56 [0.23]	0.32 [0.16]	0.15 [0.10]	5.80 [2.15]	3.62 [1.09]	3.02 [0.81]	Nil
MCL-G	71.01 [22.24]	32.57 [10.11]	–	99.90 [0.28]	92.56 [15.69]	–	4 (for Noise#2) All 30 (for Noise#3)
MCL-S	76.86 [13.12]	40.29 [4.43]	21.95 [4.79]	98.58 [0.68]	95.64 [0.77]	77.23 [1.83]	Nil

As in Env#1, the four methods were also run in Env#2 and Env#3 for different levels of injected noise. The results are summarized in Tables 2 and 3.

Scrutiny of Tables 2 and 3 reveals that the relative performances of the methods in Env#2 and Env#3 are fairly similar to that in Env#1. MCL-Y performed poorly all through. MCL-E performed poorly in Env#3 but was relatively better off in Env#2. MCL-G was fragile, as usual, when odometry noise was moderate and high, but MCL-S was robust and accurate all through. It is important to note that MCL-G failed on all 30 occasions for Noise#3 in Env#3.

For convenience of comparison, in Fig. 9 we plot the percentage of runs (out of 30) in which MCL-G and MCL-S could track the robot pose for the entire trajectory in different environments and for different levels of injected noise. From the figure it is evident that while MCL-S could track the trajectories in all cases, MCL-G succeeded in tracking complete trajectories in only a few cases. For Noise #3 in Env#3 it could not completely track even a single trajectory. Thus, MCL-S has clearly outperformed MCL-G insofar as robustness is concerned.

It may be noted that the performances of the methods are worse in Env#3 compared to their performances in Env#1 and Env#2. This is attributable to the fact that Env#3 is a long corridor. As the robot moves down the corridor, it mainly sees parallel walls. The robot can sense and ascertain its distance from these walls but is rather uncertain about its position along the length of the corridor for most part of its travel. This leads to relatively poor localization in Env#3.

In Fig. 10 we juxtapose the computation time taken by the four methods to estimate a single scan pose (i.e., to perform steps 2–7 of Fig. 1) using 200 particles. The methods were implemented in C++ and run on a PC with 2.8 GHz Intel Core 2 Duo processor and 4 GB RAM running Debian 5.0 Linux operating system. This figure

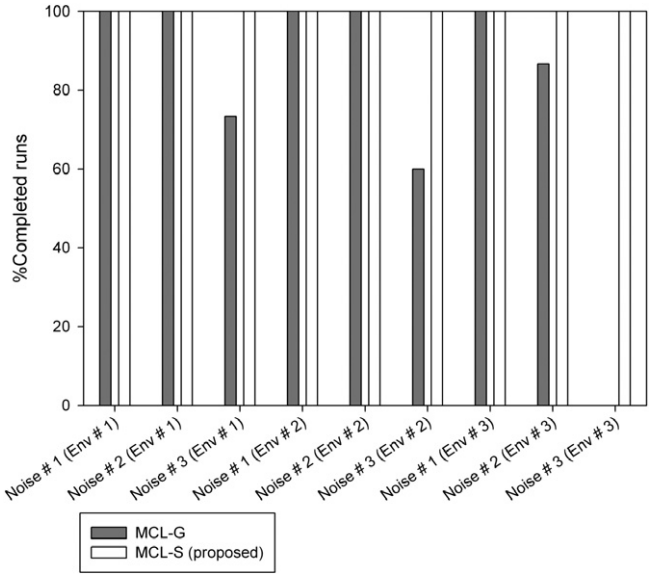


Fig. 9. Comparison of the percentage of runs (out of 30) in which MCL-G and MCL-S could complete tracking the robot pose using simulated scan data for the entire trajectory.

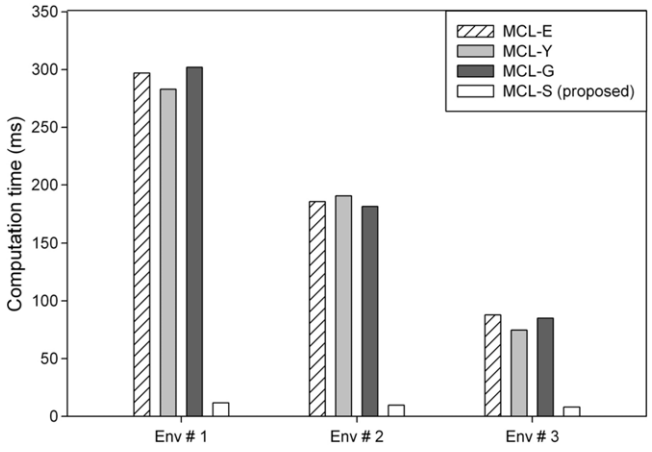


Fig. 10. Comparison of the computation times (in ms) required to estimate a single pose by the four methods using 200 particles in three different environments.

reveals that the proposed MCL-S method is computationally more efficient (by about an order of magnitude) than the other three methods. The efficiency can be attributed to the fact that, unlike the other methods, the proposed method does not perform any ray tracing, which is a very compute-intensive process. It is important to point out that in our implementation we had incorporated different strategies for speeding up the computation. We exploited the fact that, in general, only a handful of the objects in the environment will be "visible" to a particle at any given instance and thus succeeded in bringing down the computation time by about 30% for MCL-S and over 80% for the other three methods without compromising on their performances. The differences in computation time for the other three methods across different environments arise essentially due to the difference in the number of line segments in their corresponding maps. Because of a smaller computational overhead, MCL-S permits more frequent pose updates without overburdening the onboard processor. Frequent pose updates abate inflow of large odometry errors and thus help improve the accuracy of pose estimates.

We have already noted earlier that we choose the interval between two invocations of the pose estimation cycle for updating the robot pose in such a way that the robot has moved by at least 500 mm or 5°. This interval (hereinafter called *update interval*) has important ramifications. If the chosen interval is large, more odometry error will creep in and that will make accurate pose estimation more challenging. If this interval is too small, more frequent invocations of the cycle will be necessary. This will no doubt make accurate pose estimation simpler, but will leave precious little time for the CPU onboard the robot to perform other important tasks like path planning and trajectory tracking. Thus, a desirable attribute of a good localization method is that it should continue making accurate pose estimates even in the face of increased update intervals. To see how the four methods perform when confronted with different update intervals, in Fig. 11 we graph the variations in their position estimation error and orientation estimation error (mean of average position and average orientation error on 30 different trajectories) with three different update intervals using 200 particles and with injected noise corresponding to Noise#1 for Env#1. The graphs show that as the update interval increases, so does the error in position and in orientation estimate. This is natural because with an increase in the distance, translational or angular, travelled by the robot, more error gets introduced into its odometry estimate making it more uncertain about its whereabouts. From the graphs it is obvious that the performances of MCL-G and MCL-S are much better than MCL-Y and MCL-E for all the three update intervals. The performance of MCL-E deteriorates drastically with increasing update interval. The performances of MCL-G and the MCL-S are roughly at par, but while the proposed MCL-S could successfully track poses for the entire trajectory with an update interval of 1000 mm or 10° for all the three sets of noise parameters on all 30 occasions, MCL-G failed to keep track of the robot on 28 out of 30 occasions (for Noise#2) and on all the 30 occasions (for Noise#3). This clearly brings out the fragile nature of MCL-G in trying circumstances.

Now that we have compared our method with the three others, we take a closer look into our method. First, continuing with the study on the dependence of the position estimates on update intervals, we investigate into its performance using different numbers of particles. Fig. 12 shows that though a wider update interval introduces more error into the pose estimation process, an increased number of particles can offset the resulting increased uncertainty to some extent by sampling the state space more densely. For a fixed update interval and number of particles, the error in position estimate increases as the injected noise in the odometry estimate increases because it again makes the robot more uncertain about its pose. The figure corroborates the robustness of the proposed MCL-S by making it abundantly clear that MCL-S can continue to track the robot with fairly good accuracy even in the face of very high noise (viz., Noise#3) and large update interval (1000 mm or 10°). Though Fig. 12 pertains to Env#1 for Noise#3, similar graphs were obtained for Noise#1 and Noise#2, but the magnitudes of the errors were correspondingly smaller in those cases.

In Fig. 13(a) and (b), we show how the error in the estimate of each scan pose incurred by the proposed MCL-S method varies over the entire path of travel for one particular instance of the run on a trajectory generated with Noise#3 in Env#1. In Fig. 13(a) the position errors have been plotted on log scale. While the graph in red shows the error in the odometry estimate of the position, blue shows the position error incurred by MCL-S. Whereas the error in odometry grows without bound with the length of travel and becomes as high as 100 m, the error incurred by MCL-S remains mostly within 100 mm and between 100 mm and 1 m in a few cases. In Fig. 13(b), the error in the orientation estimate by odometry is shown in red while that by MCL-S is shown in blue. It is important to note that the orientation error cannot grow beyond 180°. The orientation error of MCL-S is largely within 1° and only on a few occasions it goes as high as 10° or beyond.

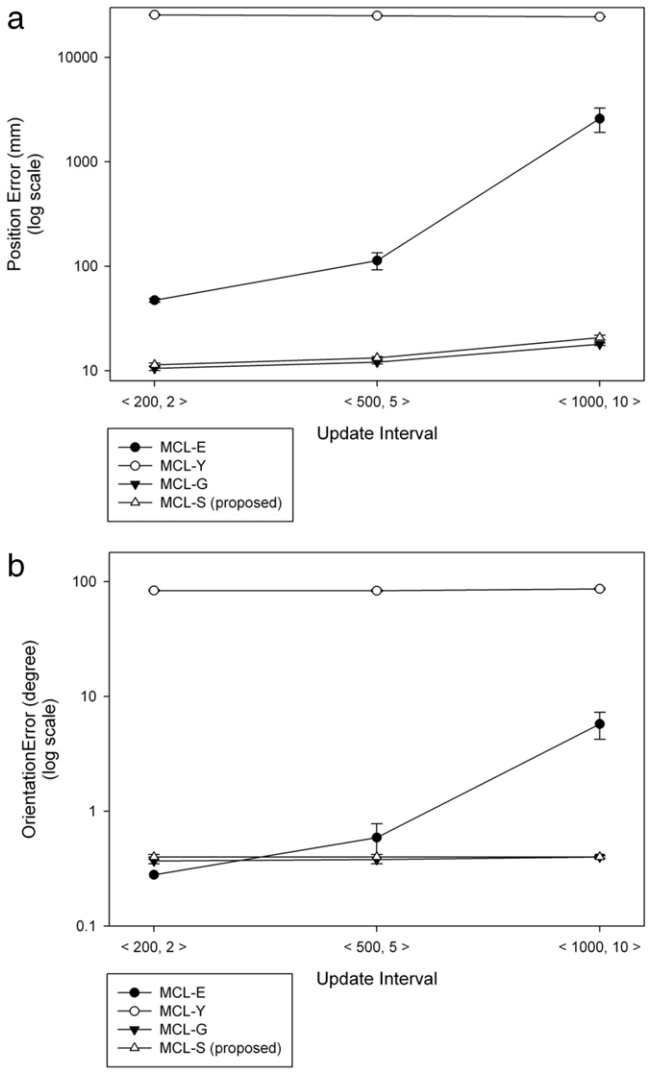


Fig. 11. (a) Mean of the average position error (in mm) in log scale versus update interval; and (b) Mean of the average orientation error (in degree) in log scale versus update interval, for Noise#1 with 200 particles in Env#1. The update intervals are denoted by a 2-tuple like ⟨500, 5⟩ with units mm and degree respectively. The update intervals used are ⟨200, 2⟩, ⟨500, 5⟩ and ⟨1000, 10⟩. The error bars indicate 95% confidence interval.

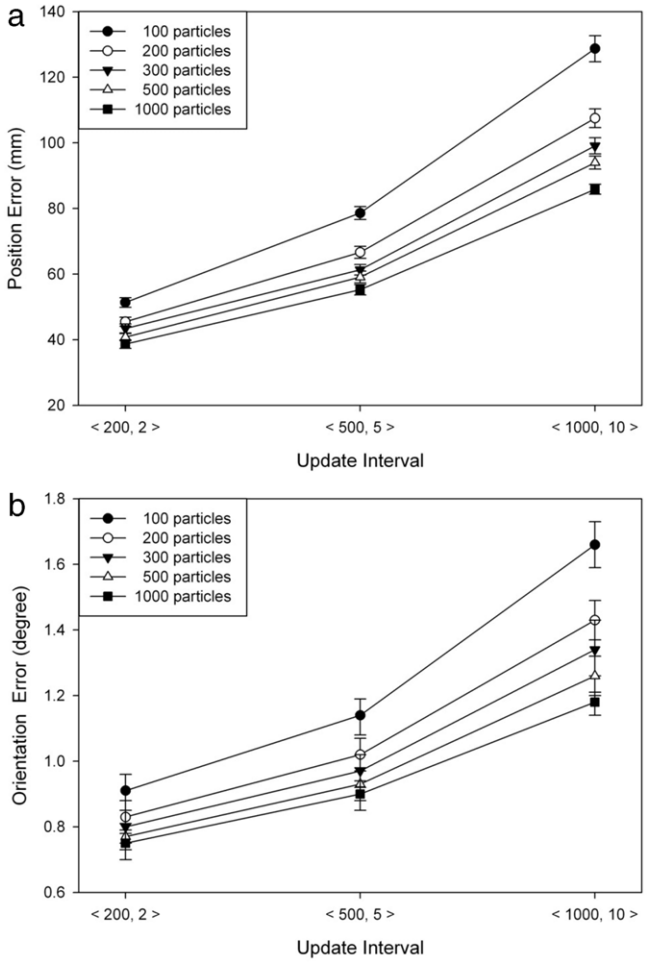


Fig. 12. Mean of the (a) average position error (in mm) versus update interval; and (b) average orientation error (in degree) versus update interval, for Noise#3 in Env#1. The update intervals are denoted by a 2-tuple like ⟨500, 5⟩ with units mm and degree respectively. The numbers of particles used are 100, 200, 300, 500 and 1000, and the update intervals are ⟨200, 2⟩, ⟨500, 5⟩ and ⟨1000, 10⟩. The error bars indicate 95% confidence interval.

(ii) *Pose estimation using real data*

In this study, we used the same maps viz., Env#1, Env#2 and Env#3 but instead of driving a simulated robot through it to generate data, we utilized the real data (consisting of odometry estimated scan poses and laser range scans) collected from the real environment while a real robot moved along a specified trajectory of approximate length 286 m, 208 m and 327 m in Env#1, Env#2 and Env#3 respectively. Since, in a real environment the true values of the scan poses are not directly available, we used the estimates of the scan poses obtained through scan matching as reasonable estimates of the corresponding true poses.

In this comparative study, we invoked each of the four methods using 200 particles and update interval of ⟨250, 10⟩ i.e., 250 mm or 10°. The mean of the average position and average orientation error of 30 runs incurred by the four methods are shown in Fig. 14.

Fig. 14 reveals that both MCL-G and MCL-S outperform the other two methods insofar as accuracy of pose tracking is concerned. But MCL-G is too fragile as it failed to track the pose along the entire trajectory on 23 out of 30 occasions in Env#1 and on 10 out of 30 occasions in Env#2. (For a graphical comparison, please see Fig. 15.) MCL-E failed too, but only on 3 occasions out of 30 in Env#2. While taking a note on the magnitudes of the mean errors, it must be kept in mind that here, unlike in the previous study using simulated data, the true poses of the robot are not accurately known. For instance, many of the scan poses estimated through scan matching (and reckoned as the true poses) did not appear plausible in Env#2 as they were seen to be outside the map when plotted. Moreover, the robot was imparted backward motion on some part of the trajectory in Env#2. These are attributable to the relatively high error in pose estimate in Env#2. Moreover, in a real environment, the map error (i.e., how closely the map models the actual environment) also introduces error in the pose estimation.

In Fig. 16, we show the trajectory of the robot estimated using odometry data (in red) and the corresponding trajectory estimated by the proposed MCL-S method (in blue). The odometry-estimated trajectory is too wayward and has moved outside the map, while the trajectory estimated using MCL-S is generally nicely confined inside the map. This study once again shows the accuracy and robustness of the proposed MCL-S method.

6. Discussion

The first step towards computing the importance weight in each of the four methods (MCL-E, MCL-Y, MCL-G and MCL-S) consists

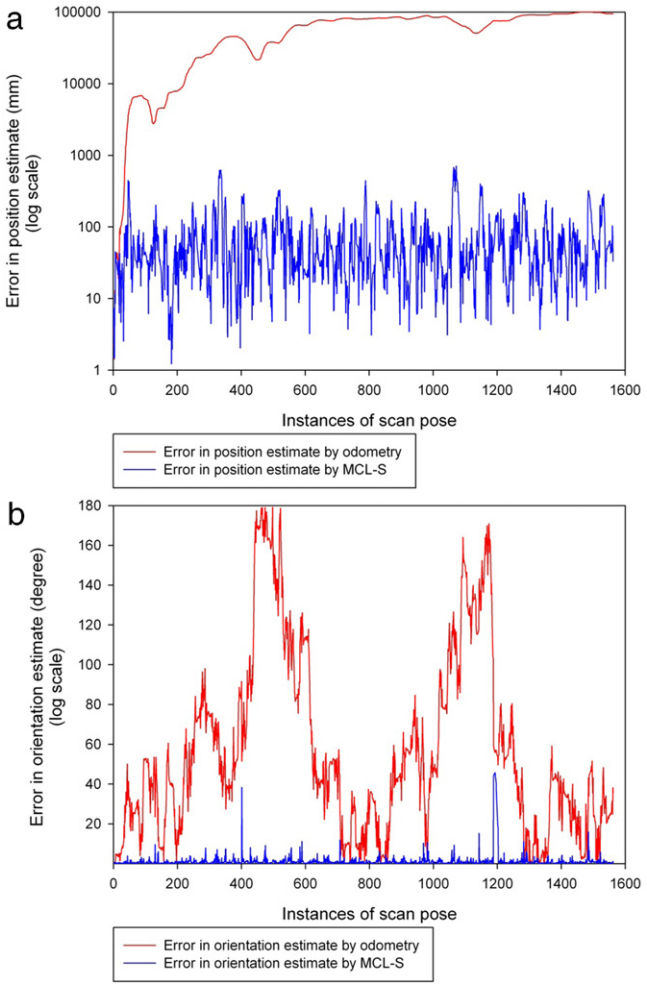


Fig. 13. (a) Position error (in mm) (in log scale); and (b) Orientation error (in degree), versus pose instances along the length of travel incurred by MCL-S. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

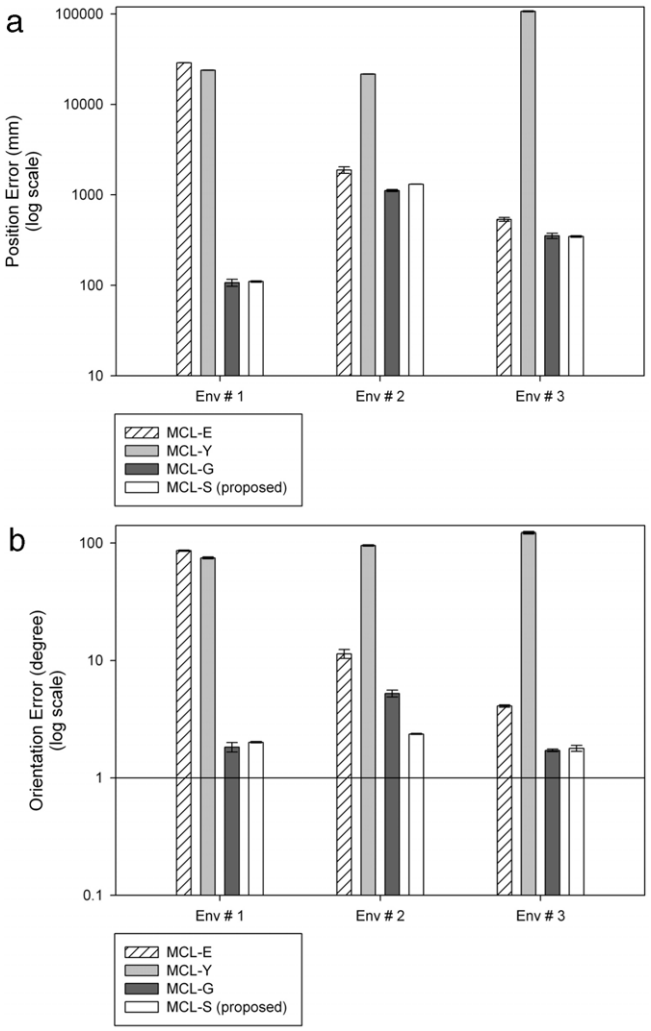


Fig. 14. Comparison of the accuracy of pose estimation using real data (a) the mean of the average position error; and (b) the mean of the average orientation error. Both the errors are plotted in log scale. The error bars indicate 95% confidence interval. MCL-G failed to track the pose for the entire trajectory on 23 out of 30 occasions in Env#1. In Env#2, MCL-G failed on 10 out of 30 runs, while MCL-E failed on 3 out of 30 runs. Only that many runs were used in computing the mean errors as could be successfully completed.

in determining the incongruity or mismatch between the actual perception of the robot from its true but unknown pose and the computed perception from the hypothesized pose represented by a particle. In MCL-Y and MCL-G, the range data acquired in a given direction is implicitly assumed to correspond to the range data obtained from the map in the same direction using ray tracing. The sum of the squared differences between all such corresponding range data gives the measure of mismatch. The correspondence holds only when the particle lies in the vicinity of the true pose. If the correspondences are incorrect, the measure of mismatch will be erroneous. MCL-E, on the other hand, does not make any explicit attempt to establish correspondence; it simply takes the difference between two sets of line segments (the OSG and the MSG, with the latter possibly rotated to roughly align with the former) as a measure of mismatch. The proposed method, in contrast to the above methods, includes a procedure for explicitly establishing the correspondence before assessing the mismatch.

The nonlinear transformation (applied on the estimate of the mismatch) governs the extent to which particles are favoured in terms of assignment of high weight values vis-à-vis their mismatch measures. In the MCL-G and the proposed method, weight assigned is the reciprocal of the square of the measure of mismatch. This makes the weight values fall off sharply with increasing mismatch, when the values of mismatch are fairly small. Such an inverse-square relationship exploits, to very high degree, the knowledge that a particle with a small measure of mismatch is close to the true pose. Since such a particle is assigned a high weight value, it shepherds the remaining particles in the state space and makes a dense cluster in its vicinity, leaving few particles elsewhere. The dense accumulation of particles in the correct region of the state space enables accurate estimation of the robot pose. However, if the measure of mismatch turns out to be substantially in error in the sense that particles that have been assigned small mismatch values are actually far away from the true pose, the particles converge in the wrong region of the state space and the pose estimate becomes erroneous. In an extreme situation, no particle may be present in the region of the state space where the robot is actually present and thus pose tracking may fail completely. Since the procedure employed by MCL-G for assessing the mismatch depends on the correctness of the correspondences, which is not very robust, and at the same time uses an inverse-square transformation, the procedure is prone to failures. As seen in Section 5, this is the reason behind the occasional failures of MCL-G though it otherwise gave accurate pose estimates. MCL-Y employs an identical procedure for mismatch assessment as MCL-G, but it does not use any transformation of the mismatch values. It simply normalizes the mismatch values and reckons these values

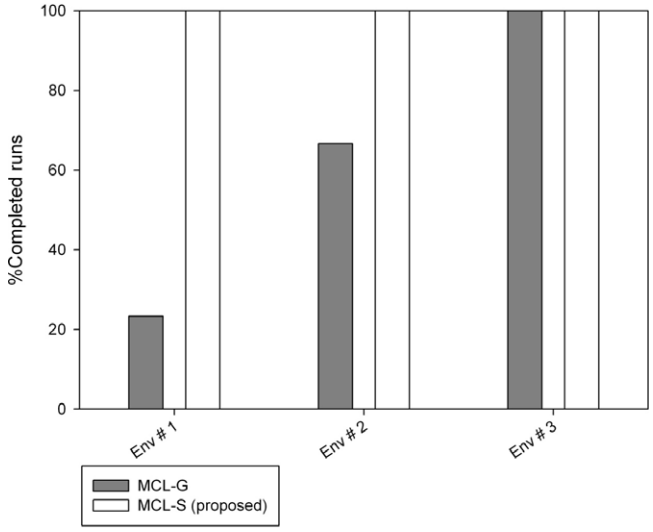


Fig. 15. Comparison of the percentage of runs (out of 30) in which MCL-G and MCL-S could complete tracking the robot pose using real scan data for the entire trajectory.

as the corresponding weights. (Strictly speaking, therefore, step 4 of Fig. 1 is superfluous for this method). Since this method does not exploit, to an appreciable extent, the information of the particles with high weights, particles do not densely cluster around the region of the state space where the robot is possibly situated after the resampling step. Thus, this method fails to give an accurate estimate of the pose. For the same reason, it never fails as a few particles always remain scattered over the map. The nonlinear transformation that MCL-E uses, does exploit the information associated with particles with high weights, but not to the extent (which is regulated by I_p) that MCL-G or the proposed method does. Moreover, its methodology for mismatch assessment is also not very accurate. Thus, so far as pose tracking is concerned, it is less accurate than both MCL-G and MCL-S, but somewhat more robust than MCL-G. However, if a small value of I_p like $I_p = 0.2$ is used, the performance of MCL-E improves for low noise and small update interval, but deteriorates substantially for high noise and large update intervals. Moreover, if MCL-E is modified so as to use the same nonlinear transformation that MCL-G and the proposed method use, its accuracy in pose tracking improves but it becomes more fragile.

The above discussion, considered in the backdrop of the simulation and the experimental results, makes it abundantly clear that the accuracy and robustness of a pose tracking method depend both on the method adopted to assess the mismatch and the nonlinear transformation used. Since our method uses a transformation function that falls off sharply with the extent of

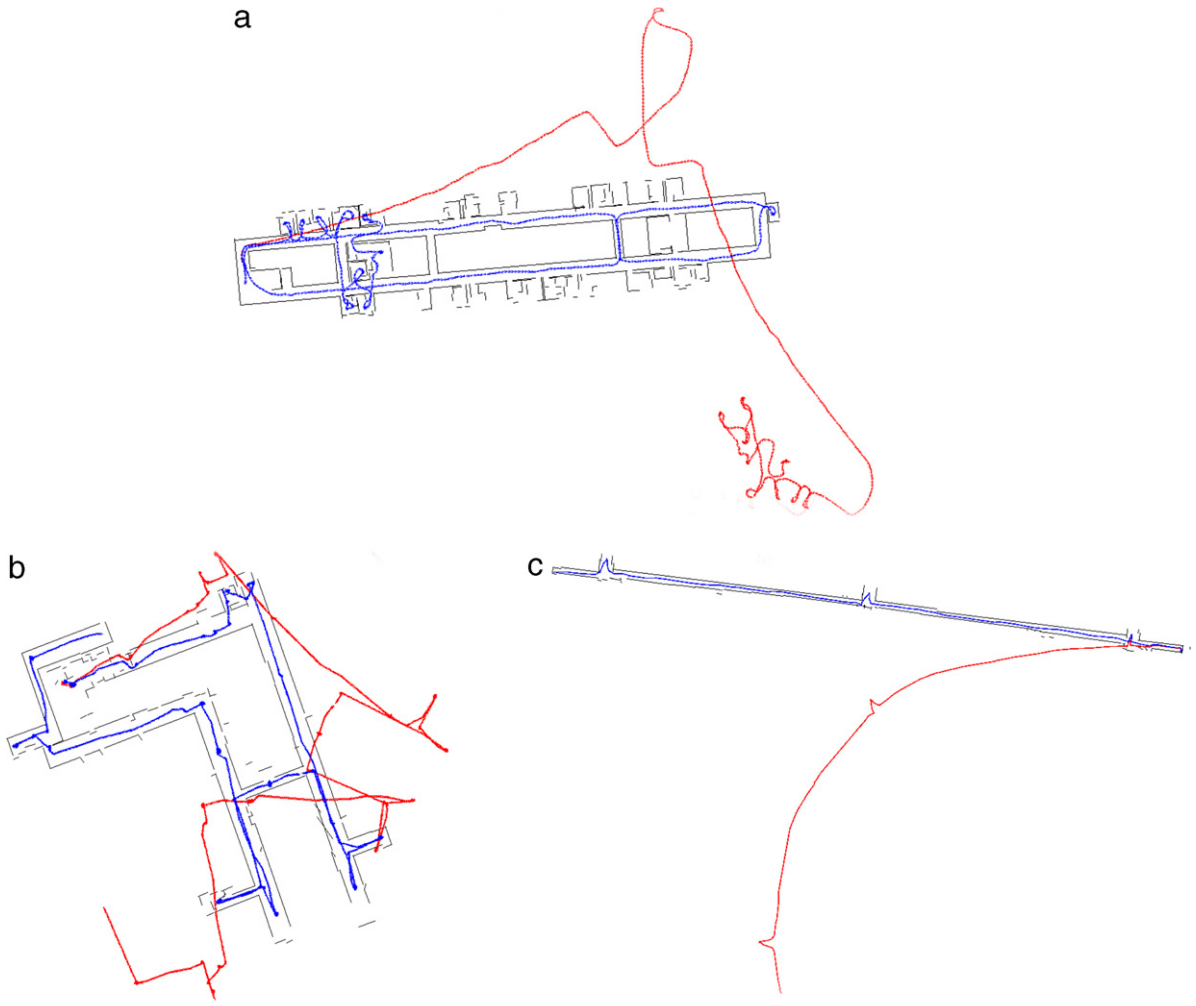


Fig. 16. The trajectory estimated using real odometry data is shown in red, while the trajectory estimated by the proposed MCL-S method using real odometry and range data is shown in blue in (a) Env#1; (b) Env#2; and (c) Env#3. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

mismatch and is buttressed by a robust and accurate method of assessing the mismatch, we have been able to achieve accurate and robust pose tracking all through.

7. Conclusion

The particle filtering framework on which MCL is based, keeps open before the programmer the choice of strategy to be adopted for computing the importance weights associated with the particles. And it is this strategy that, generally speaking, endows a particular implementation of MCL with attributes like robustness, accuracy and efficiency or robs it of them. In this paper, we have proposed a heuristic-based method for computing the importance weights of an MCL in the context of pose tracking on line segment-based maps and have, through extensive studies using both simulated and real data, corroborated that the method is both accurate and robust in addition to being computationally efficient. Furthermore, the proposed method proved to be superior to three other methods of weight computation [7,10,11] insofar as endowing MCL with all the above attributes are concerned.

The synergy resulting from the twin uses of a reliable mismatch assessment procedure and an acutely nonlinear transformation function, endows an MCL using the proposed method of weight computation with the capability to continuously track the pose of a robot over fairly long paths in a very accurate, robust and computationally efficient manner. It is to be specifically kept in mind that the proposed method has performed well on maps that have not been handcrafted or produced by CAD software and that are not free of 'noisy' features.

Preliminary studies revealed that the proposed method of weight computation holds promise for global localization as well. The particles, initially dispersed all over the map, were found to converge in the vicinity of the robot within a few time steps and correctly track the robot pose thereafter. However, extensive simulation and experimental studies will have to be carried out before making any claim as to its usefulness for global localization. The problem of premature convergence that arises often in MCL will also have to be looked into [48]. Moreover, a method for efficient generation of particles such that they are uniformly dispersed but confined inside the map in the initial state also has to be formulated. Investigating into and addressing these issues could be an area of future work.

Acknowledgements

The first author wishes to thank Debranjan Sarkar for his guidance and encouragement. The code for processing laser range scans used in this work was obtained from Steffen Gutmann many years ago. We thank him for sharing the code with us. The comments of the two anonymous reviewers were of immense help in improving the quality of this paper. The public data, on which our method was tested, were obtained from the Robotics Data Set Repository (Radish) [47]. Thanks go to Nakju Lett Doh, Gian Maria Pelusi and Regis Vincent for providing the data.

References

[1] H.P. Moravec, A. Elfes, High resolution maps from wide angle sonar, in: Proceedings of the IEEE International Conference on Robotics and Automation, ICRA, 1985, pp. 116–121.
[2] B. Sarkar, P.K. Pal, D. Sarkar, Building maps of indoor environments by merging line segments extracted from registered laser range scans, Robot. Auton. Syst. 62 (2014) 603–615.
[3] F. Amigoni, M. Vailati, A method for reducing redundant line segments in maps, in: Proceedings of Fourth European Conference on Mobile Robots, ECMR, 2009, pp. 61–66.
[4] R. Lakaemper, Simultaneous multi-line-segment merging for robot mapping using Mean shift clustering, in: Proceedings of IEEE/RSJ International Conference on Intelligent Robots and Systems, IROS, 2009, pp. 1654–1660.
[5] F. Dellaert, D. Fox, W. Burgard, S. Thrun, Monte Carlo localization for mobile robots, in: Proceedings of the IEEE International Conference on Robotics and Automation, ICRA, 1999, pp. 1322–1328.
[6] J.-S. Gutmann, D. Fox, An experimental comparison of localization methods continued, in: Proceedings of IEEE/RSJ International Conference on Intelligent Robots and Systems, IROS, 2002, pp. 454–459.
[7] P. Espinace, A. Soto, M. Torres-Torriti, Real-time robot localization in indoor environments using structural information, in: Proceedings of IEEE Latin American Robotic Symposium, LARS 2008, pp. 79–84.
[8] A. Gasparri, S. Panzieri, F. Pascucci, G. Ulivi, Monte Carlo filter in mobile robotics localization: A clustered evolutionary point of view, J. Intell. Robot. Syst. 47 (2006) 155–174.
[9] T. He, S. Hirose, A global localization approach based on Line-segment Relation Matching technique, Robot. Auton. Syst. 60 (2012) 95–112.
[10] T. Yaqub, J. Katupitiya, Laser scan matching for measurement update in a particle filter, in: Proceedings of the IEEE/ASME International Conference on Advanced Intelligent Mechatronics, 2007, pp. 4–7.
[11] A. Gasparri, S. Panzieri, F. Pascucci, G. Ulivi, Genetic approach for a localisation problem based upon particle filters, in: Proceedings of 8th International IFAC Symposium on Robot Control, SYROCO, 2006.
[12] S. Thrun, Probabilistic algorithms in robotics, AI Mag. 21 (2000) 93–109.
[13] S. Thrun, W. Burgard, D. Fox, Probabilistic Robotics, MIT Press, 2005.
[14] J. Leonard, H. Durrant-Whyte, Mobile robot localization by tracking geometric beacons, IEEE Trans. Robot. Autom. 7 (3) (1991) 376–382.
[15] F. Donoso-Aguirre, J.-P. Bustos-Salas, M. Torres-Torriti, A. Guesalaga, Mobile robot localization using the Hausdorff distance, Robotica 26 (2008) 129–141.
[16] W. Burgard, D. Fox, D. Hennig, T. Schmidt, Estimating the absolute position of a mobile robot using position probability grids, in: Proceedings of the Fourteenth National Conference on Artificial Intelligence, 1996, pp. 896–901.
[17] W. Burgard, D. Fox, D. Hennig, Fast grid-based position tracking for mobile robots, in: Proceedings of the 21th German Conference on Artificial Intelligence (KI), 1997, Springer Verlag, Freiburg, Germany, 1997.
[18] W. Burgard, A. Derr, D. Fox, A.B. Cremers, Integrating global position estimation and position tracking for mobile robots: the dynamic Markov localization approach, in: Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems, IROS, 1998, pp. 730–735.
[19] J.-S. Gutmann, W. Burgard, D. Fox, K. Konolige, An experimental comparison of localization methods, in: Proceedings of IEEE/RSJ International Conference on Intelligent Robots and Systems, IROS, 1998, pp. 736–743.
[20] J.-S. Gutmann, Markov–Kalman localization for mobile robots, in: Proceedings of the 16th International Conference on Pattern Recognition, 2002, pp. 601–604.
[21] M.S. Arulampalam, S. Maskell, N. Gordon, T. Clapp, A tutorial on particle filters for online nonlinear/non-Gaussian Bayesian tracking, IEEE Trans. Signal Process. 50 (2002) 174–188.
[22] D. Fox, S. Thrun, F. Dellaert, W. Burgard, Particle filters for mobile robot localization, in: A. Doucet, N. de Freitas, N. Gordon (Eds.), Sequential Monte Carlo Methods in Practice, Springer Verlag, New York, 2000.
[23] S. Thrun, D. Fox, W. Burgard, F. Dellaert, Robust Monte Carlo localization for mobile robots, Artificial Intelligence 128 (2001) 99–141.
[24] T.-B. Kwon, J.-H. Yang, J.-B. Song, W. Chung, Efficiency improvement in Monte Carlo localization through topological information, in: Proceedings of IEEE/RSJ International Conference on Intelligent Robots and Systems, IROS, 2006, pp. 424–429.
[25] J.-S. Lee, W.K. Chung, Robust mobile robot localization in highly non-static environments, Auton. Robots 29 (2010) 1–16.
[26] H. Kose, H.L. Akın, The Reverse Monte Carlo localization algorithm, Robot. Auton. Syst. 55 (2007) 480–489.
[27] J. Rowekamper, C. Sprunk, G.D. Tipaldi, C. Stachniss, P. Pfaff, W. Burgard, On the position accuracy of mobile robot localization based on particle filters combined with scan matching, in: Proceedings of IEEE/RSJ International Conference on Intelligent Robots and Systems, IROS, 2012, pp. 3158–3164.
[28] L. Moreno, J.M. Armingol, S. Garrido, A. De La Escalera, M.A. Salichs, A genetic algorithm for mobile robot localization using ultrasonic sensors, J. Intell. Robot. Syst. 34 (2002) 135–154.
[29] L. Ronghua, H. Bingrong, Coevolution based adaptive Monte Carlo localization (CEAMCL), Int. J. Adv. Robot. Syst. 1 (2004) 183–190.
[30] M. Lisowski, F. Zhun, O. Ravn, Differential evolution to enhance localization of mobile robots, in: Proceedings of IEEE International Conference on Fuzzy Systems, 2011, pp. 241–247.
[31] L. Moreno, S. Garrido, M.L. Munoz, Evolutionary filter for robust mobile robot localization, Robot. Auton. Syst. 54 (2006) 590–600.
[32] F. Martín, L. Moreno, D. Blanco, M.L. Muñoz, Kullback–Leibler divergence-based global localization for mobile robots, Robot. Auton. Syst. 62 (2014) 120–130.
[33] M. Bashiri, H. Vatankhah, S.S. Ghidary, Hybrid adaptive differential evolution for mobile robot localization, Intell. Serv. Robot. 5 (2012) 99–107.
[34] A.R. Vahdat, N. NourAshrafoddin, S.S. Ghidary, Mobile robot global localization using differential evolution and particle swarm optimization, in: IEEE Congress on Evolutionary Computation, CEC, 2007, pp. 1527–1534.

[35] M.-P. Dubuisson, A.K. Jain, A modified Hausdorff distance for object matching, in: Proceedings of the 12th IAPR International Conference on Pattern Recognition, Vol. 1. 1994, pp. 566–568.
[36] S. Saha, B. Sarkar, P.K. Pal, Monte Carlo-based pose tracking on maps represented with line segments, in: Advances in Robotics—2015, 2nd International Conference of the Robotics Society of India, Goa, India, 2015, http://dx.doi.org/10.1145/2783449.2783512 (in press).
[37] C. Guerra, V. Pascucci, Line-based object recognition using Hausdorff distance: from range images to molecular secondary structures, Image Vis. Comput. 23 (2005) 405–415.
[38] O. Wulf, B. Wagner, M. Khalaf-Allah, Using 3D data for Monte Carlo localization in complex indoor environments, in: Proceedings of the 2nd Bi-Annual European Conference on Mobile Robots, ECMR, 2005, pp. 170–175.
[39] J.E. Baker, Reducing bias and inefficiency in the selection algorithm, in: Proceedings of the Second International Conference on Genetic Algorithms and their Application, 1987, pp. 14–21.
[40] N.J. Gordon, D.J. Salmond, A.F.M. Smith, Novel approach to nonlinear/non-Gaussian Bayesian state estimation, in: IEE Proceedings F on Radar and Signal Processing, 1993, pp. 107–113.
[41] A.F.M. Smith, A.E. Gelfand, Bayesian statistics without tears: A sampling-resampling perspective, Amer. Statist. 46 (1992) 84–88.
[42] D.B. Rubin, Using the SIR algorithm to simulate posterior distributions, in: J.M. Bernardo, M.H. DeGroot, D.V. Lindley, A.F.M. Smith (Eds.), Bayesian Statistics—3, Oxford University Press, 1988.
[43] V. Nguyen, S. Gächter, A. Martinelli, N. Tomatis, R. Siegwart, A comparison of line extraction algorithms using 2D range data for indoor mobile robotics, Auton. Robots 23 (2007) 97–111.
[44] J.R. Beveridge, E.M. Riseman, How easy is matching 2D line models using local search? IEEE Trans. Pattern Anal. Mach. Intell. 19 (1997) 564–579.
[45] M. Mazuran, F. Amigoni, Matching line segment scans with mutual compatibility constraints, in: Proceedings of the IEEE International Conference on Robotics and Automation, ICRA, 2014, pp. 4298–4303.
[46] F. Amigoni, M. Reggiani, V. Schiaffonati, Simulations used as experiments in autonomous mobile robotics, in: IEEE International Conference on Robotics and Automation, ICRA, Workshop on "The Role of Experiments in Robotics Research", 2010.
[47] A. Howard, N. Roy, The robotics data set repository (Radish), 2003. http://radish.sourceforge.net/.
[48] G. Kootstra, B. de Boer, Tackling the premature convergence problem in Monte-Carlo localization, Robot. Auton. Syst. 57 (2009) 1107–1118.



Biswajit Sarkar is a Scientific Officer at Variable Energy Cyclotron Centre, Kolkata, India since 1998. He joined the Training School of Bhabha Atomic Research Centre, Mumbai, India in 1996 after completing his Bachelors degree course in Electrical Engineering from Jalpaiguri Government Engineering College, West Bengal, India in the same year. During the last 15 years, apart from developing various software packages related to the operation of cyclotron, he was involved in research in the areas of computer animation, soft computing and mobile robot navigation. He is a Post Graduate Teacher of the Homi Bhabha National Institute, India and is involved in teaching at the Training School of Bhabha Atomic Research Centre, Mumbai.



Surojit Saha is a Scientific Officer in the Variable Energy Cyclotron Centre, Kolkata, India. He joined the Training School of Bhabha Atomic Research Centre, Mumbai in 2009 after completing his B-Tech. in Computer Science & Engineering from West Bengal University of Technology, Kolkata, India in the same year. He received his M-Tech degree in Computer Engineering from Homi Bhabha National Institute, Mumbai, India in the year 2012. He has been working in the field of robotics (indoor mobile robots) for the last couple of years. His research is primarily focused into localization and path tracking of mobile robots in indoor environments.



Prabir K. Pal is Head, Autonomous Robotics Section at the Division of Remote Handling & Robotics of Bhabha Atomic Research Centre, Mumbai, India and a Professor of the Homi Bhabha National Institute, a deemed University, India. He joined the Training School of Bhabha Atomic Research Centre in 1978 after receiving his Master's degree in Physics from the University of Calcutta, India, in 1976. He obtained his Ph.D. from the University of Mumbai, India, in 1997. Over the last 30 years, he has worked mainly in the areas of Robot Motion Planning, Gait Generation for Walking Machines, and Mobile Robot Navigation. He has also been steering projects in Vision Guided and Force Controlled Robot Manipulation, Telepresence and Medical Robotics.